SIMATS SCHOOL OF ENGINEERING

Approved By AICTE | IET-UK Accreditation

Course Code: DSA03 11	Course Name: Natural Language Processing
Branch: AI&DS	Year: 2024-2028
Name	A. Abhishek Reddy
Rg no	192424192

Assignment 1

1. Design an FSA that recognizes simple date expressions like March 15, the 22nd of November, Christmas. You should try to include all such "absolute" dates, (e.g. not "deictic" ones relative to the current day like the day before yesterday). Each edge of the graph should have a word or a set of words on it. You should use some sort of shorthand for classes of words to avoid drawing too many arcs (e.g., furniture→ desk, chair, table). (10M) (CO1) (BLT4)

2. Write regular expressions for the following languages: You may use either Perl notation or the minimal "algebraic" notation of Sec. 2.3, but make sure to say which one you are using. By "word", we mean an alphabetic string separated from other words by white space, any relevant punctuation, line breaks, and so forth. (10M) (CO1) (BLT4)

a. the set of all alphabetic strings.

b. the set of all lowercase alphabetic strings ending in a b.

c. the set of all strings with two consecutive repeated words (e.g., "Humbert Humbert" and "the the" but not "the bug" or "the big bug").

d. the set of all strings from the alphabet a,b such that each a is immediately preceded and immediately followed by a b.

e. all strings that start at the beginning of the line with an integer and that end at the end of the line with a word.

f. all strings which have both the word grotto and the word raven in them. (but not, for example, words like grottos that merely contain the word grotto).

g. write a pattern which places the first word of an English sentence in a register.

Deal with punctuation.

3. Computing minimum edit distances by hand, figure out whether drive is closer to brief or to divers, and what the edit distance is. You may use any version of distance that you like. (10M) (CO1) (BLT4)

QUESTION 1: DESIGN AN FSA THAT RECOGNIZES SIMPLE DATE EXPRESSIONS

1.1 Introduction

Finite State Automata (FSA) is a mathematical model used in Natural Language Processing to recognize patterns in a sequence of words or symbols. It consists of a set of states and transitions between those states.

FSA reads the input from left to right and checks whether the given input follows a valid pattern. It is useful for recognizing dates, numbers, words and other language structures.

Date recognition is an important application of NLP because dates are commonly used in documents, messages, emails, calendars and event descriptions.

1.2 Objective

1. To design a Finite State Automaton for recognizing absolute date expressions.
2. To identify different date formats using word categories.
3. To recognize month names, day numbers, ordinal numbers and holidays.
4. To accept valid date expressions and reject invalid expressions.
5. To understand how FSA is used in NLP applications.

1.3 Basic Components of FSA

A Finite State Automaton contains five important components.

Component	Description

States	Different conditions of the automaton
Input alphabet	Words or symbols given as input
Start state	The state where processing begins
Transitions	Movement from one state to another
Final state	State that indicates successful recognition

1.4 Types of Date Expressions

Absolute date expressions represent fixed dates or calendar occasions.

Examples:

- March 15
- 15 March
- The 22nd of November
- December 25
- Christmas
- January 1, 2025
- Independence Day

Relative date expressions depend on the current day and are not included in this FSA.

Examples:

- Today
- Tomorrow
- Yesterday
- Next week
- The day before yesterday

1.5 Word Categories Used in FSA

To reduce the number of transitions, similar words are grouped into classes.

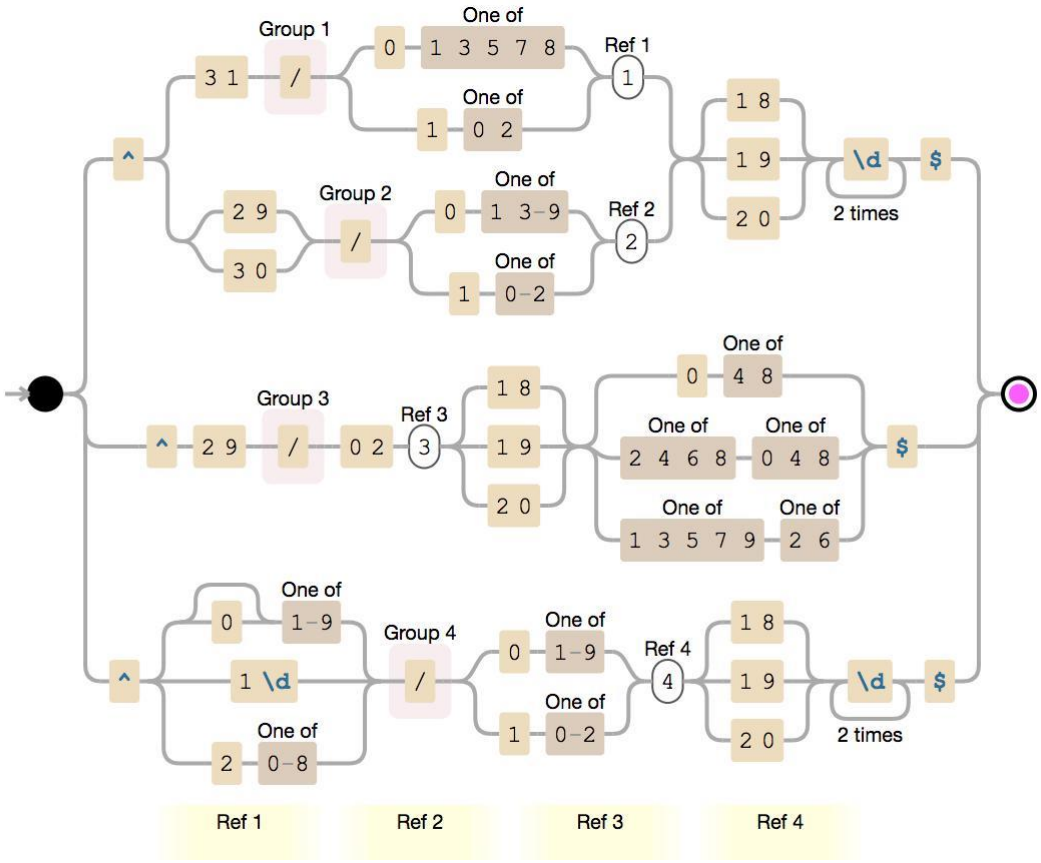
Word Class	Examples
MONTH	January, February, March, April, May, June, July, August, September, October, November, December
DAY	1, 2, 3, 4, 5, ..., 31
ORDINAL	1st, 2nd, 3rd, 4th, ..., 31st
ARTICLE	The
PREPOSITION	Of
YEAR	2024, 2025, 2026
HOLIDAY	Christmas, New Year's Day, Independence Day

1.6 Common Date Formats

The FSA recognizes the following date structures.

Date Format	Example
Month + Day	March 15
Day + Month	15 March
Article + Ordinal + Of + Month	The 22nd of November
Month + Day + Year	March 15, 2025
Holiday	Christmas
Day + Month + Year	15 March 2025

1.7 FSA State Diagram



The FSA begins with the START state and checks the input tokens. If the tokens follow a valid date structure, the automaton reaches the ACCEPT state.

Main transition:

START → MONTH → DAY → YEAR (Optional) → ACCEPT

Alternative transitions are provided for day-first dates, ordinal dates and holiday names.

1.8 FSA Transition Table

Current State	**Input**	**Next State**
START	MONTH	MONTH_SEEN
MONTH_SEEN	DAY	DAY_SEEN
DAY_SEEN	YEAR or End	ACCEPT
START	ARTICLE	ARTICLE_SEEN

ARTICLE_SEEN	ORDINAL	ORDINAL_SEEN
ORDINAL_SEEN	PREPOSITION	OF_SEEN
OF_SEEN	MONTH	ACCEPT
START	HOLIDAY	ACCEPT

1.9 Working of FSA

Example: The 22nd of November.

Step	**Input**	**Word Class**	**State**
1	The	ARTICLE	ARTICLE_SEEN
2	22nd	ORDINAL	ORDINAL_SEEN
3	Of	PREPOSITION	OF_SEEN
4	November	MONTH	ACCEPT

The input follows the correct sequence and reaches the final state. Therefore, the expression is accepted.

1.10 Accepted and Rejected Expressions

Input	**Result**	**Reason**
March 15	Accepted	Valid date
15 March	Accepted	Valid date
The 22nd of November	Accepted	Valid ordinal date
Christmas	Accepted	Fixed holiday
March 15, 2025	Accepted	Date with year
Tomorrow	Rejected	Relative date
March March	Rejected	Invalid structure

The of November	Rejected	Missing ordinal
February 31	Rejected	Invalid calendar date

1.11 Applications of FSA in NLP

1. FSA is used to identify dates in documents and sentences.
2. It helps extract examination dates and event dates.
3. It is useful in calendar and appointment applications.
4. It helps recognize fixed holidays and important occasions.
5. It improves text processing by identifying valid word sequences.

1.12 Advantages of FSA

1. It is simple and easy to understand.
2. It recognizes patterns quickly.
3. It reduces complex language structures into states and transitions.
4. It supports different date formats.
5. It can be used in information extraction systems.

1.13 Limitations of FSA

1. It cannot understand the complete meaning of a sentence.
2. It requires predefined date patterns.
3. It may require additional validation for impossible dates.
4. It does not automatically recognize every possible date format.
5. Complex language expressions may require advanced NLP techniques.

Python Program:

```
import re

months = (
   "January|February|March|April|May|June|July|August|"
```

```
    "September|October|November|December"
)
holidays = [
    "Christmas",
    "New Year's Day",
    "New Year"
]
pattern1 = rf"(?:{months})\s+\d{{1,2}}(?:st|nd|rd|th)?"
pattern2 = rf"the\s+\d{{1,2}}(?:st|nd|rd|th)\s+of\s+(?:{months})"
def check_date(date):
    if re.fullmatch(pattern1, date, re.IGNORECASE):
        return "Accepted: Absolute Date"
    if re.fullmatch(pattern2, date, re.IGNORECASE):
        return "Accepted: Absolute Date"
    if date.lower() in [h.lower() for h in holidays]:
        return "Accepted: Holiday Date"
    return "Rejected: Invalid Date Expression"
date = input("Enter a date expression: ")
print(check_date(date))
```

OUTPUT:

```
Enter a date expression: March 15
Accepted: Absolute Date
```

1.14 Result

A Finite State Automaton was designed to recognize absolute date expressions such as March 15, 15 March, the 22nd of November and Christmas. Different word classes and transitions were used to identify valid date structures and reject relative date expressions.

QUESTION 2: WRITE REGULAR EXPRESSIONS FOR THE FOLLOWING LANGUAGES

2.1 Introduction to Regular Expressions

A Regular Expression (RE) is a sequence of characters used to identify, search and match specific patterns in text. It is an important technique in Natural Language Processing.

Regular expressions are used to identify words, numbers, punctuation marks, repeated words and sentence structures.

They help computers process large amounts of text quickly and accurately.

2.2 Objectives

1. To understand the concept of regular expressions.
2. To identify different patterns in English text.
3. To write regular expressions for the given seven problems.
4. To handle spaces, punctuation and word boundaries.
5. To understand the applications of regular expressions in NLP.

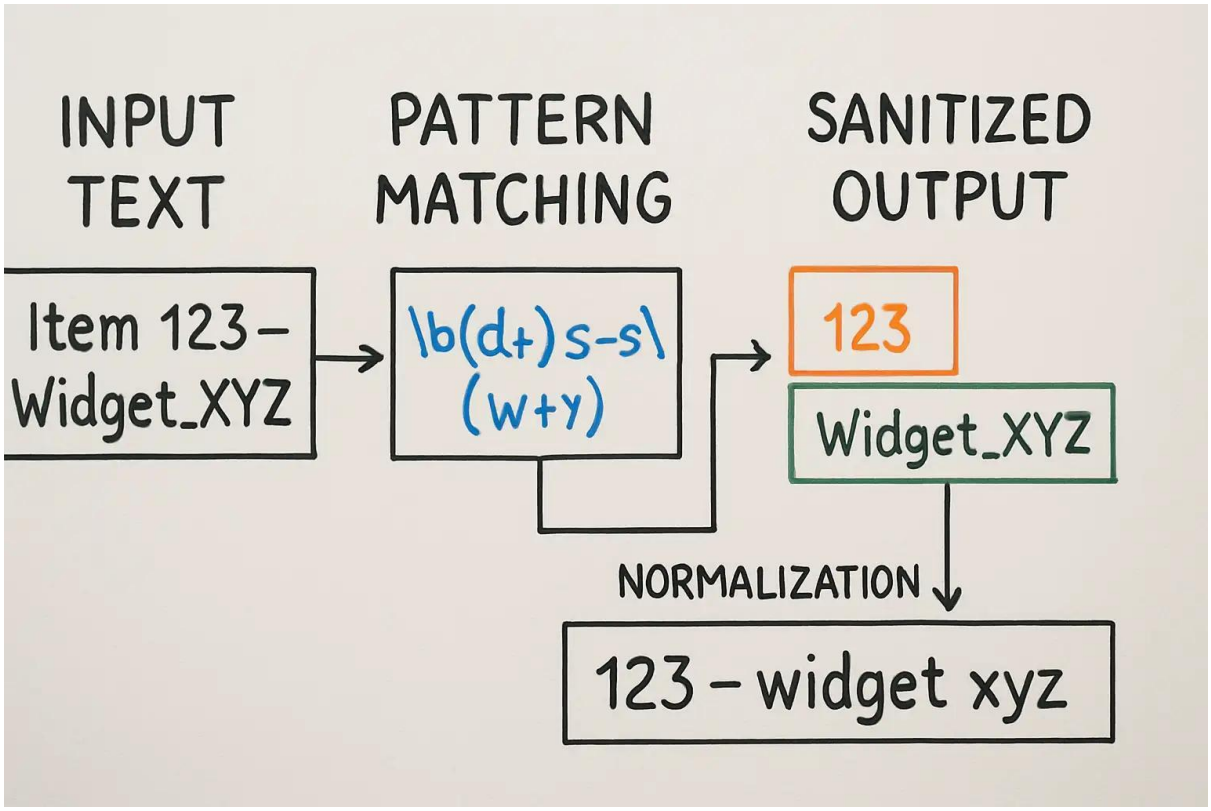
2.3 Regular Expression Symbols

The assignment uses Perl-style regular expression notation.

Symbol	Meaning	Example
^	Beginning of a line	^Hello
$	End of a line	World$
[A-Z]	Uppercase letters	[A-Z]+

[a-z]	Lowercase letters	[a-z]+
[A-Za-z]	Uppercase and lowercase letters	[A-Za-z]+
[0-9]	Digits	[0-9]+
+	One or more repetitions	a+
*	Zero or more repetitions	a*
?	Optional character or group	a?
\b	Word boundary	\bgrotto\b
\s	Whitespace	\s+
\d	Digit	\d+
|	Alternative	cat|dog
( )	Grouping	(abc)
\1	Backreference to first group	([a-z]+)\s+\1

2.4 Regular Expression Processing Diagram



Explanation:

1. Input text is given to the system.
2. The text is divided into words and characters.
3. A regular expression pattern is applied.
4. The system checks whether the pattern matches.
5. The matching result is displayed or extracted.

2.5 (a) The Set of All Alphabetic Strings

Question: Write a regular expression to identify all alphabetic strings.

Regular Expression:

^[A-Za-z]+$

Explanation:

- ^ represents the beginning of the string.
- [A-Za-z] accepts uppercase and lowercase letters.

- + represents one or more characters.
- $ represents the end of the string.

This expression accepts strings containing only alphabetic characters.

Examples:

Input	Output	Reason
Hello	Match	Contains letters
NLP	Match	Uppercase letters
abcXYZ	Match	Both cases
hello123	No Match	Contains numbers
good day	No Match	Contains space
hello@	No Match	Contains symbol

Result: The regular expression successfully identifies all alphabetic strings.

2.6 (b) Lowercase Alphabetic Strings Ending in b

Question: Write a regular expression for all lowercase alphabetic strings ending with the letter b.

Regular Expression:

^[a-z]*b$

Explanation:

- ^ represents the beginning.
- [a-z] accepts lowercase letters.
- * allows zero or more lowercase letters.
- b ensures that the string ends with b.
- $ represents the end.

Examples:

Input	Output	Reason
cab	Match	Ends with b
ab	Match	Lowercase and ends with b
b	Match	Single-letter string
abc	No Match	Ends with c
Cab	No Match	Contains uppercase letter
apple	No Match	Does not end with b

Result: The expression identifies lowercase alphabetic strings ending in the letter b.

2.7 (c) Strings with Two Consecutive Repeated Words

Question: Identify strings containing two consecutive repeated words, such as "Humbert Humbert" and "the the".

Regular Expression:

\b([A-Za-z]+)\b\s+\1\b

Explanation:

- \b identifies the beginning and ending of a word.
- ([A-Za-z]+) captures the first word.
- \s+ identifies one or more spaces.
- \1 checks whether the next word is the same as the first word.
- The final \b ensures a complete word match.

Examples:

Input	Output	Reason
Humbert Humbert	Match	Same word repeated
the the	Match	Same word repeated

cat cat	Match	Consecutive repetition
the bug	No Match	Different words
the big bug	No Match	Not consecutive repetition
cat, cat	Requires punctuation-aware pattern	Comma separates the words

To allow punctuation between repeated words, use:

\b([A-Za-z]+)\b[\s\p{Punct}]+\1\b

Result: The regular expression identifies consecutive repeated words using capturing groups and backreferences.

2.8 (d) Strings from the Alphabet a,b Where Each a Is Surrounded by b

Question: Identify strings containing only a and b, where every a is immediately preceded and followed by b.

Regular Expression:

^b+(?:ab+)*$

Explanation:

- ^ represents the beginning of the string.
- b+ ensures that the string starts with b.
- (?:ab+)* allows a to appear only when followed by b.
- $ represents the end of the string.

Every occurrence of a must have b on both sides.

Examples:

Input	**Output**	**Reason**
b	Match	Contains no a
bb	Match	Contains only b

bab	Match	a is surrounded by b
bbabb	Match	a has b on both sides
a	No Match	a is not surrounded
ab	No Match	No preceding b
ba	No Match	No following b

Result: The regular expression accepts valid strings where every a is surrounded by b.

2.9 (e) Strings Starting with an Integer and Ending with a Word

Question: Identify lines that start with an integer and end with an alphabetic word.

Regular Expression:

^\s*\d+\b.*\b[A-Za-z]+[.!?]?\s*$

Explanation:

- ^ represents the beginning of the line.
- \s* allows optional spaces at the beginning.
- \d+ identifies one or more digits.
- \b identifies the word boundary.
- .* accepts the middle content.
- [A-Za-z]+ identifies the final alphabetic word.
- [.!?]? allows optional sentence punctuation.
- $ represents the end of the line.

Examples:

Input	**Output**	**Reason**
25 students attended	Match	Starts with integer and ends with word
2025 is the year.	Match	Starts with number and ends with year

There are 25 students	No Match	Does not start with integer
25 students!	Match	Ends with word and punctuation
25	No Match	No alphabetic ending

Result: The regular expression identifies lines beginning with an integer and ending with an alphabetic word.

2.10 (f) Strings Containing Both grotto and raven

Question: Identify all strings that contain both the complete words "grotto" and "raven".

Regular Expression:

^(?=.*\bgrotto\b)(?=.*\braven\b).*$

Explanation:

- ^ represents the beginning of the string.
- (?=.*\bgrotto\b) checks whether the complete word grotto exists.
- (?=.*\braven\b) checks whether the complete word raven exists.
- .* accepts any characters between the words.
- $ represents the end of the string.

The pattern accepts both words in any order.

Examples:

Input	**Output**	**Reason**
The grotto contains a raven.	Match	Both words present
A raven entered the grotto.	Match	Both words present
The grottos are dark; raven flies.	No Match	grottos is not grotto
Only the grotto is shown.	No Match	raven is missing
A raven is nearby.	No Match	grotto is missing

Result: The regular expression identifies strings containing both complete words without matching larger words.

2.11 (g) Pattern to Place the First Word of an English Sentence in a Register

Question: Write a pattern that captures the first word of an English sentence in a register.

A register is a storage location used to hold a value. In regular expressions, a capturing group can store the required word.

Regular Expression:

(?:^|(?<=[.!?])\s+)[\s"'“‘(]*([A-Za-z]+(?:[-'][A-Za-z]+)*)

Explanation:

- ^ identifies the beginning of the text.
- [.!?] represents sentence-ending punctuation.
- \s+ identifies the space after punctuation.
- [A-Za-z]+ identifies the first alphabetic word.
- (...) captures the first word.
- (?:[-'][A-Za-z]+)* allows internal hyphens or apostrophes.

The first word is stored in capturing group 1.

Examples:

Input	**Captured Word**
Hello there. NLP is useful.	Hello, NLP
Natural language is useful.	Natural
Wait! Stop now.	Wait, Stop
Students learn NLP.	Students

Result: The regular expression captures the first word of a sentence and stores it in a register using a capturing group.

2.12 Handling Punctuation in Regular Expressions

Punctuation marks are important when identifying words and sentences. A regular expression must consider commas, full stops, question marks, quotation marks and other symbols.

Punctuation	Example	Purpose
Comma	Hello, world	Separates words
Full stop	NLP is useful.	Ends a sentence
Question mark	How are you?	Ends a question
Exclamation mark	Excellent!	Shows emotion
Quotation marks	"Natural Language"	Encloses text
Hyphen	Well-known	Joins words

The \b symbol helps identify complete words, while [.!?] identifies common sentence-ending punctuation.

2.13 Comparison of All Seven Regular Expressions

Part	Language requirement	Regular expression
a	All alphabetic strings	^[A-Za-z]+$
b	Lowercase strings ending in b	^[a-z]*b$
c	Two repeated consecutive words	\b([A-Za-z]+)\b\s+\1\b
d	Each a surrounded by b	^b+(?:ab+)*$
e	Integer at beginning and word at end	^\s*\d+\b.*\b[A-Za-z]+[.!?]?\s*$
f	Both grotto and raven	^(?=.*\bgrotto\b)(?=.*\braven\b).*$
g	Capture first sentence word	(?:^|(?<=[.!?])\s+)[\s"'“‘(]*([A-Za-z]+(?:[-'][A-Za-z]+)*)

2.14 Advantages of Regular Expressions

1. They identify text patterns quickly.
2. They reduce manual searching in large documents.
3. They help extract important information.

4. They support text cleaning and preprocessing.
5. They are useful for validating words and character sequences.

2.15 Applications of Regular Expressions in NLP

1. Identifying dates and numbers in documents.
2. Detecting repeated words in sentences.
3. Extracting names and keywords.
4. Checking spelling and text formatting.
5. Identifying sentence beginnings and endings.

Python Program

```
import re

def check_a(s):
    return bool(re.fullmatch(r"[A-Za-z]+", s))

def check_b(s):
    return bool(re.fullmatch(r"[a-z]*b", s))

def check_c(s):
    return bool(re.search(r"\b(\w+)\b\s+\1\b", s, re.IGNORECASE))

def check_d(s):
    return bool(re.fullmatch(r"b+(?:ab+)*", s))

def check_e(s):
    return bool(re.fullmatch(r"\d+\b.*\b[A-Za-z]+", s))

def check_f(s):
    return bool(
        re.search(r"\bgrotto\b", s, re.IGNORECASE)
        and re.search(r"\braven\b", s, re.IGNORECASE)
```

```
    )
def check_g(s):
    return re.findall(
        r"(?:^|[.!?]\s+)[\"“‘(]*([A-Za-z]+(?:[-'][A-Za-z]+)*)",
        s
    )
print("a:", check_a("Hello"))
print("b:", check_b("ab"))
print("c:", check_c("hello hello"))
print("d:", check_d("bbabb"))
print("e:", check_e("123 students"))
print("f:", check_f("The grotto contains a raven."))
print("g:", check_g("Hello everyone. Welcome to NLP."))
```

OUTPUT:

```
a: True
b: True
c: True
d: True
e: True
f: True
g: ['Hello', 'Welcome']
```

2.16 Result

Seven regular expressions were designed using Perl-style notation. Each expression identifies a specific language pattern, including alphabetic strings, repeated words, lowercase strings,

integer-based lines, complete words and sentence beginnings. Punctuation and word boundaries were also considered to improve pattern recognition.

QUESTION 3: COMPUTING MINIMUM EDIT DISTANCES BY HAND (10 MARKS)

3.1 Introduction

Minimum Edit Distance (MED) is a technique used in Natural Language Processing to calculate the minimum number of operations required to transform one word or string into another.

It is commonly used in spelling correction, text comparison, search engines and language processing applications.

The three basic operations are insertion, deletion and substitution.

3.2 Objectives

1. To understand the concept of minimum edit distance.
2. To identify insertion, deletion and substitution operations.
3. To calculate the edit distance between two words.
4. To compare the similarity between drive, brief and divers.
5. To understand the applications of edit distance in NLP.

3.3 Types of Edit Operations

Operation	**Meaning**	**Example**
Insertion	Adding a character	Cat → Cart
Deletion	Removing a character	Plane → Plan
Substitution	Replacing a character	Cat → Cut
Match	Keeping the same character	Drive → Drive

In standard Levenshtein distance, each insertion, deletion and substitution has a cost of 1. A matching character has a cost of 0.

3.4 Minimum Edit Distance Algorithm

The minimum edit distance can be calculated using dynamic programming.

Let:

- X = Source word
- Y = Target word
- D(i,j) = Minimum cost of transforming the first i characters of X into the first j characters of Y.

The recurrence formula is:

$$ D(i,j)=\min \begin{cases} D(i-1,j)+1 \\ D(i,j-1)+1 \\ D(i-1,j-1)+c \end{cases} $$

Where:

$$ c= \begin{cases} 0 & \text{if characters are equal} \\ 1 & \text{if characters are different} \end{cases} $$

The final value in the bottom-right corner of the matrix gives the minimum edit distance.

3.5 Steps for Calculating Edit Distance

1. Write the source word horizontally or vertically.
2. Write the target word along the other direction.
3. Create a matrix with one extra row and column.
4. Initialize the first row and column with increasing numbers.
5. Calculate each cell using insertion, deletion and substitution.
6. Select the smallest value.
7. Read the bottom-right value as the final edit distance.

3.6 First Comparison: DRIVE to BRIEF

Source word: DRIVE

Target word: BRIEF

Both words contain five characters.

Compare each character:

Position	DRIVE	BRIEF	Operation
1	D	B	Substitution
2	R	R	Match
3	I	I	Match
4	V	E	Substitution
5	E	F	Substitution

3.7 Transformation Process

The source word DRIVE can be changed into BRIEF using three substitutions.

Step	Word	Operation	Cost
1	DRIVE	Original word	0
2	BRIVE	D → B	1
3	BRIEE	V → E	1
4	BRIEF	E → F	1

Total cost:

$$ 1+1+1=3 $$

Therefore,

$$ \boxed{D(\text{DRIVE},\text{BRIEF})=3} $$

3.8 Edit Distance Matrix: DRIVE to BRIEF

The dynamic programming matrix is shown below.

	ε	B	R	I	E	F
ε	0	1	2	3	4	5

D	1	1	2	3	4	5
R	2	2	1	2	3	4
I	3	3	2	1	2	3
V	4	4	3	2	2	3
E	5	5	4	3	2	3

The final matrix value is 3.

Edit distance = 3

3.9 Second Comparison: DRIVE to DIVERS

Source word: DRIVE

Target word: DIVERS

The source contains five characters, and the target contains six characters.

The transformation can be performed using one deletion and two insertions.

3.10 Transformation Process

Step	Word	Operation	Cost
1	DRIVE	Original word	0
2	DIVE	Delete R	1
3	DIVER	Insert R	1
4	DIVERS	Insert S	1

Total cost:

$$ 1+1+1=3 $$

Therefore,

$$ \boxed{D(\text{DRIVE},\text{DIVERS})=3} $$

3.11 Edit Distance Matrix: DRIVE to DIVERS

	ε	D	I	V	E	R	S
ε	0	1	2	3	4	5	6
D	1	0	1	2	3	4	5
R	2	1	1	2	3	3	4
I	3	2	1	2	3	4	4
V	4	3	2	1	2	3	4
E	5	4	3	2	1	2	3

The final value in the matrix is 3.

Edit distance = 3

3.12 Comparison of Both Edit Distances

The two comparisons are performed using the same Levenshtein distance method.

Comparison	Source	Target	Minimum Edit Distance
First	DRIVE	BRIEF	3
Second	DRIVE	DIVERS	3

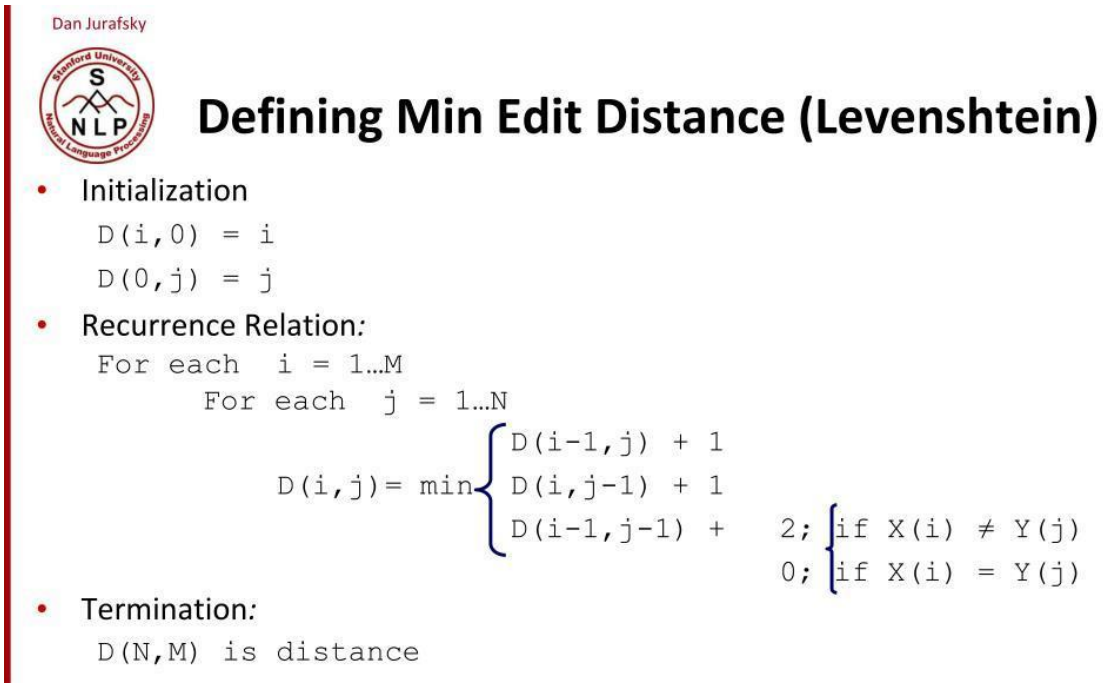
Both words have the same minimum edit distance from DRIVE.

3.13 Difference Between the Two Transformations

Operation	DRIVE → BRIEF	DRIVE → DIVERS
Insertion	0	2
Deletion	0	1
Substitution	3	0
Total cost	3	3

The first comparison requires three substitutions. The second comparison requires one deletion and two insertions.

3.14 Edit Distance Flowchart



Explanation:

1. Start with the source and target words.
2. Initialize the distance matrix.
3. Compare the characters.
4. Calculate insertion, deletion and substitution costs.
5. Select the minimum cost for each cell.
6. Continue until all characters are processed.
7. Display the final minimum edit distance.

3.15 Applications of Minimum Edit Distance

1. Spell Checking: Finds spelling mistakes and suggests correct words.
2. Search Engines: Identifies words that are similar to the search input.
3. Text Correction: Helps correct typing mistakes in documents.

4. OCR Systems: Corrects errors in text extracted from scanned images.
5. NLP Applications: Measures the similarity between words and strings.

3.16 Advantages of Minimum Edit Distance

1. It is simple and easy to calculate.
2. It identifies the minimum number of editing operations.
3. It helps improve spelling correction systems.
4. It supports approximate string matching.
5. It is useful for comparing words and text sequences.

3.17 Limitations of Minimum Edit Distance

1. It does not understand the meaning of words.
2. Different operations may have different importance in real language.
3. Similarity is based on characters rather than pronunciation.
4. Large strings require more computation.
5. It may produce multiple possible transformations with the same cost.

Python Program

```
def edit_distance(word1, word2):
    m = len(word1)
    n = len(word2)
    dp = [[0 for j in range(n + 1)] for i in range(m + 1)]
    for i in range(m + 1):
        dp[i][0] = i
    for j in range(n + 1):
        dp[0][j] = j
    for i in range(1, m + 1):
```

```
        for j in range(1, n + 1):
            if word1[i - 1] == word2[j - 1]:
                cost = 0
            else:
                cost = 1
            dp[i][j] = min(
                dp[i - 1][j] + 1,
                dp[i][j - 1] + 1,
                dp[i - 1][j - 1] + cost
            )
    return dp[m][n]
d1 = edit_distance("DRIVE", "BRIEF")
d2 = edit_distance("DRIVE", "DIVERS")
print("Distance between DRIVE and BRIEF:", d1)
print("Distance between DRIVE and DIVERS:", d2)
if d1 < d2:
    print("BRIEF is closer to DRIVE")
elif d2 < d1:
    print("DIVERS is closer to DRIVE")
else:
    print("Both words have equal distance from DRIVE")
```

Output:

```
Distance between DRIVE and BRIEF: 3
Distance between DRIVE and DIVERS: 3
Both words have equal distance from DRIVE
```

3.18 Result

The minimum edit distance was calculated between DRIVE and BRIEF, and between DRIVE and DIVERS using the Levenshtein distance method.

- DRIVE to BRIEF = 3
- DRIVE to DIVERS = 3

Therefore, both BRIEF and DIVERS are equally close to DRIVE according to standard Levenshtein distance.

OVERALL CONCLUSION

This assignment explains three important techniques used in Natural Language Processing: Finite State Automata, Regular Expressions and Minimum Edit Distance.

The first question demonstrates how FSA recognizes absolute date expressions using states, transitions and word categories. It identifies valid date formats such as March 15, 15 March, the 22nd of November and Christmas.

The second question explains seven regular expression patterns used to recognize alphabetic strings, repeated words, lowercase strings, specific character sequences, integers, complete words and sentence beginnings. It also explains the importance of punctuation and word boundaries.

The third question calculates minimum edit distance using insertion, deletion and substitution operations. The distance between DRIVE and BRIEF is 3, and the distance between DRIVE and DIVERS is also 3.

These three techniques provide a foundation for text recognition, information extraction, text correction and other NLP applications.

REFERENCES

1. Daniel Jurafsky and James H. Martin, Speech and Language Processing, 3rd Edition, Draft.
2. Christopher D. Manning and Hinrich Schütze, Foundations of Statistical Natural Language Processing, MIT Press, 1999.
3. Steven Bird, Ewan Klein and Edward Loper, Natural Language Processing with Python, O'Reilly Media, 2009.
4. Jeffrey E. F. Friedl, Mastering Regular Expressions, 3rd Edition, O'Reilly Media, 2006.
5. Thomas H. Cormen et al., Introduction to Algorithms, 3rd Edition, MIT Press, 2009.